

GETTING STARTED WITH

Terminal, Git & GitHub

A quick reference guide to the commands you'll use every day

Welcome to the team

This guide covers the terminal basics, core Git commands, and the GitHub workflow you'll need to get up and running. Keep it handy — nobody memorizes this on day one.

Contents

- 1. Terminal Basics — moving around and managing files
- 2. Git Basics — tracking changes to your code
- 3. Working with Branches
- 4. GitHub — working with remote repositories
- 5. Everyday Workflow Cheat Sheet
- 6. Good Habits & Common Pitfalls

1. Terminal Basics

The terminal (also called the command line or shell) lets you navigate your computer and run programs by typing commands instead of clicking. Here are the ones you'll use constantly.

Navigation

Command	What it does
<code>pwd</code>	Print working directory — shows the folder you're currently in.
<code>ls</code>	List the files and folders in the current directory.
<code>ls -la</code>	List all files, including hidden ones, with details (size, permissions, date).
<code>cd <folder></code>	Change directory — move into <folder>.
<code>cd ..</code>	Move up one directory (to the parent folder).
<code>cd ~</code>	Jump straight to your home directory.
<code>cd -</code>	Jump back to the previous directory you were in.

Files and Folders

Command	What it does
<code>mkdir <name></code>	Make a new directory (folder) called <name>.
<code>touch <file></code>	Create a new empty file.
<code>rm <file></code>	Remove (delete) a file. There is no recycle bin — be careful!
<code>rm -r <folder></code>	Remove a folder and everything inside it, recursively.
<code>mv <src> <dest></code>	Move or rename a file/folder.
<code>cp <src> <dest></code>	Copy a file. Use <code>cp -r</code> for folders.
<code>cat <file></code>	Print the contents of a file to the screen.
<code>clear</code>	Clear the terminal screen.

Tip: Press the Up arrow to cycle through previous commands, and Tab to auto-complete file and folder names — it saves a lot of typing.

2. Git Basics

Git is a version control system — it tracks changes to your code over time so you can review history, undo mistakes, and collaborate without overwriting each other's work. Think of it as a very detailed "save history" for your project.

Setting Up & Starting a Project

Command	What it does
<code>git init</code>	Turn the current folder into a Git repository (starts tracking it).
<code>git clone <url></code>	Download a copy of an existing repository (e.g. from GitHub) to your computer.
<code>git config --global user.name "Name"</code>	Set the name that will be attached to your commits.
<code>git config --global user.email "you@email.com"</code>	Set the email attached to your commits.

Tracking Changes

Command	What it does
<code>git status</code>	Show which files have changed, been added, or are untracked. Use this constantly.
<code>git add <file></code>	Stage a specific file — mark it to be included in the next commit.
<code>git add .</code>	Stage all changed files in the current folder.
<code>git commit -m "message"</code>	Save the staged changes as a new commit, with a short message describing them.
<code>git diff</code>	Show the exact line-by-line changes that haven't been staged yet.
<code>git log</code>	Show the history of commits, newest first.
<code>git log --oneline</code>	Show a condensed, one-line-per-commit version of the history.

Tip: A commit is a snapshot of your project at a point in time. Commit often, and write messages that explain **why**, not just **what**.

3. Working with Branches

Branches let you work on a feature or fix without touching the main codebase until it's ready. The default branch is usually called main.

Command	What it does
<code>git branch</code>	List all branches in the repository; the current one is marked with <code>*</code> .
<code>git branch <name></code>	Create a new branch called <code><name></code> .
<code>git checkout <name></code>	Switch to the branch called <code><name></code> .
<code>git checkout -b <name></code>	Create a new branch and switch to it in one step.
<code>git switch <name></code>	Modern alternative to checkout for switching branches.
<code>git merge <name></code>	Merge the branch <code><name></code> into the branch you're currently on.
<code>git branch -d <name></code>	Delete a branch that's already been merged.

Tip: Create a new branch for every feature or bug fix (e.g. `fix-login-bug`). Keep `main` clean and stable.

4. GitHub — Working with Remote Repositories

GitHub hosts your Git repositories online so you can back them up, share them, and collaborate with others. A "remote" is just a version of your repository stored elsewhere, usually on GitHub.

Command	What it does
<code>git remote -v</code>	Show the remote repositories linked to your local project.
<code>git remote add origin <url></code>	Connect your local repository to a GitHub repository.
<code>git push</code>	Upload your committed changes to the remote repository (GitHub).
<code>git push -u origin <branch></code>	Push a branch to GitHub for the first time and link it for future pushes.
<code>git pull</code>	Download and merge changes from the remote repository into your local one.
<code>git fetch</code>	Download changes from the remote without merging them yet — lets you review first.

Key GitHub Concepts

- Repository ("repo") — a project's folder, tracked by Git, hosted on GitHub.
- Fork — your own copy of someone else's repository, under your account.
- Pull Request (PR) — a request to merge your branch's changes into another branch, with a space for review and discussion.

- Clone vs Fork — clone copies a repo to your computer; fork copies it to your own GitHub account.
- Issues — GitHub's built-in tool for tracking bugs, tasks, and feature requests.

5. Everyday Workflow Cheat Sheet

This is roughly the sequence you'll repeat most days once you're working on a shared project:

Command	What it does
<code>git pull</code>	Get the latest changes before you start working.
<code>git checkout -b my-feature</code>	Create a branch for your work.
<code>(edit files)</code>	Make your changes in your code editor.
<code>git status</code>	Check what changed.
<code>git add .</code>	Stage your changes.
<code>git commit -m "describe change"</code>	Save a snapshot with a clear message.
<code>git push -u origin my-feature</code>	Upload your branch to GitHub.
Open a Pull Request	Ask for your branch to be reviewed and merged, on GitHub.com.

6. Good Habits & Common Pitfalls

- Run `git status` often — it tells you exactly where you stand before you do anything risky.
- Commit small, logical chunks rather than one giant commit at the end of the day.
- Never commit passwords, API keys, or secrets — use a `.gitignore` file to exclude sensitive or unnecessary files.
- Pull before you push, so you're working from the latest version and avoid conflicts.
- If you're unsure what a command will do, ask before running it — some actions (like `rm -r` or `git push --force`) can't be undone.
- Write commit messages a stranger could understand six months from now.

That's it for the basics — everything else you'll pick up by doing. Don't hesitate to ask questions; everyone was new to this once.